



Санкт-Петербургский Национальный Исследовательский Университет ИТМО
Факультет Программной Инженерии и Компьютерной Техники

Расчётно графическая работа № 1

По дисциплине
Теория функций комплексного переменного

Выполнил студент группы Р3219:
Ануфриев Андрей
Бегун Фёдор
Торопов Павел

Преподаватель:
Поздняков Семен Сергеевич

Санкт-Петербург
2025г.

Содержание

1	Свойства 1 и 2 для множества Мандельброта	2
1.1	Свойство 1	2
1.2	Свойство 2	2
2	Визуализация множества Мандельброта	4
2.1	Код	4
2.2	Картинки	5
3	Визуализация заполненного множества Жюлиа	11
3.1	Код	11
3.2	Картинки	12
4	Magnet fractals Type II	18
4.1	Описание	18
4.2	Код	18
4.3	Картинки	19
5	Вывод	24

1 Свойства 1 и 2 для множества Мандельброта

1.1 Свойство 1

Утверждение: Если $c \in M$, то и $\bar{c} \in M$, где M — множество Мандельброта, а $\bar{c}$ — комплексно-сопряжённое к c .

Доказательство:

1. Напомним, что множество Мандельброта M — это множество таких $c \in \mathbb{C}$, что последовательность

$$z_0 = 0, \quad z_{n+1} = z_n^2 + c$$

остаётся ограниченной, то есть существует $R > 0$, что $\forall n$ выполнено $|z_n| < R$.

2. Рассмотрим сопряжённое комплексное число $\bar{c}$. Построим последовательность (w_n) с $w_0 = 0$:

$$w_{n+1} = w_n^2 + \bar{c}$$

Докажем, что $w_n = \bar{z}_n$ для любого n по индукции:

- *База:* $n = 0$, $w_0 = 0 = \bar{0} = \bar{z}_0$.
- *Переход:* Пусть $w_n = \bar{z}_n$ для некоторого n . Тогда: $w_{n+1} = w_n^2 + \bar{c}$
 $= (\bar{z}_n)^2 + \bar{c}$
 $= \overline{z_n^2 + c}$
 $= \overline{z_{n+1}}$ Следовательно, $w_{n+1} = \bar{z}_{n+1}$.

3. Таким образом, по индукции $w_n = \bar{z}_n$ для любого n .
4. Значит, $|w_n| = |\bar{z}_n| = |z_n|$ (модуль числа не меняется при сопряжении!). Следовательно, ограниченность последовательности (z_n) эквивалентна ограниченности (w_n) .
5. Итак, $c \in M \iff \bar{c} \in M$, т.е. M симметрично относительно вещественной оси.

Почему последовательность z_n уходит в бесконечность при $|z_n| > 2$:

Если на каком-то шаге m выполнено $|z_m| > 2$, то для любого $n \geq m$: $|z_{n+1}| = |z_n^2 + c| \geq ||z_n|^2 - |c||$. Поскольку $|z_n| > 2$, то $|z_n|^2 > 4$. Если выбрать $|z_n|$ настолько большим, что $|z_n|^2 > 2|c|$, то

$$|z_{n+1}| \geq |z_n|^2 - |c| > \frac{1}{2}|z_n|^2,$$

то есть $|z_{n+1}|$ строго больше, чем $|z_n|$, и перестаёт быть ограниченным сверху. Следовательно, начиная с этого момента, последовательность модулей $|z_n|$ при каждом переходе хотя бы удваивается, стремясь к бесконечности. Поэтому если $|z_n|$ хоть раз превысит 2, уже далее орбита обязательно покинет любой круг конечного радиуса и не вернётся внутрь него.

1.2 Свойство 2

Утверждение: Если $|c| > 2$, то c не принадлежит множеству Мандельброта.

Доказательство:

1. Пусть $|c| > 2$. Построим последовательность (z_n) :

$$z_0 = 0, \quad z_{n+1} = z_n^2 + c$$

2. Уже на первом шаге $z_1 = c$, поэтому $|z_1| = |c| > 2$.
3. Покажем, что если на каком-либо шаге $|z_n| > 2$, то на следующем шаге обязательно $|z_{n+1}| > |z_n|$.
4. Заметим, что для любого $a, b \in \mathbb{C}$:

$$|a + b| \geq ||a| - |b||$$

Тогда, учитывая $z_{n+1} = z_n^2 + c$: $|z_{n+1}| = |z_n^2 + c|$
 $\geq ||z_n|^2 - |c||$

5. Для $|z_n| > 2$, рассмотрим $|z_n|^2 - |c|$: ведь $|z_n| > |c| > 2$, т.е. $|z_n|^2 - |c| > |c|^2 - |c| > 2^2 - |c| \geq 4 - |c|$. Но $|c| > 2$, поэтому $|z_{n+1}| > 2$ и, более того, $|z_{n+1}| > |z_n|$.
6. Следовательно, начиная с z_1 , все последующие члены последовательности будут по модулю строго больше 2 и расти неограниченно, то есть последовательность уходит на бесконечность.
7. Значит, при $|c| > 2$ последовательность не может остаться ограниченной, то есть $c \notin M$.

2 Визуализация множества Мандельброта

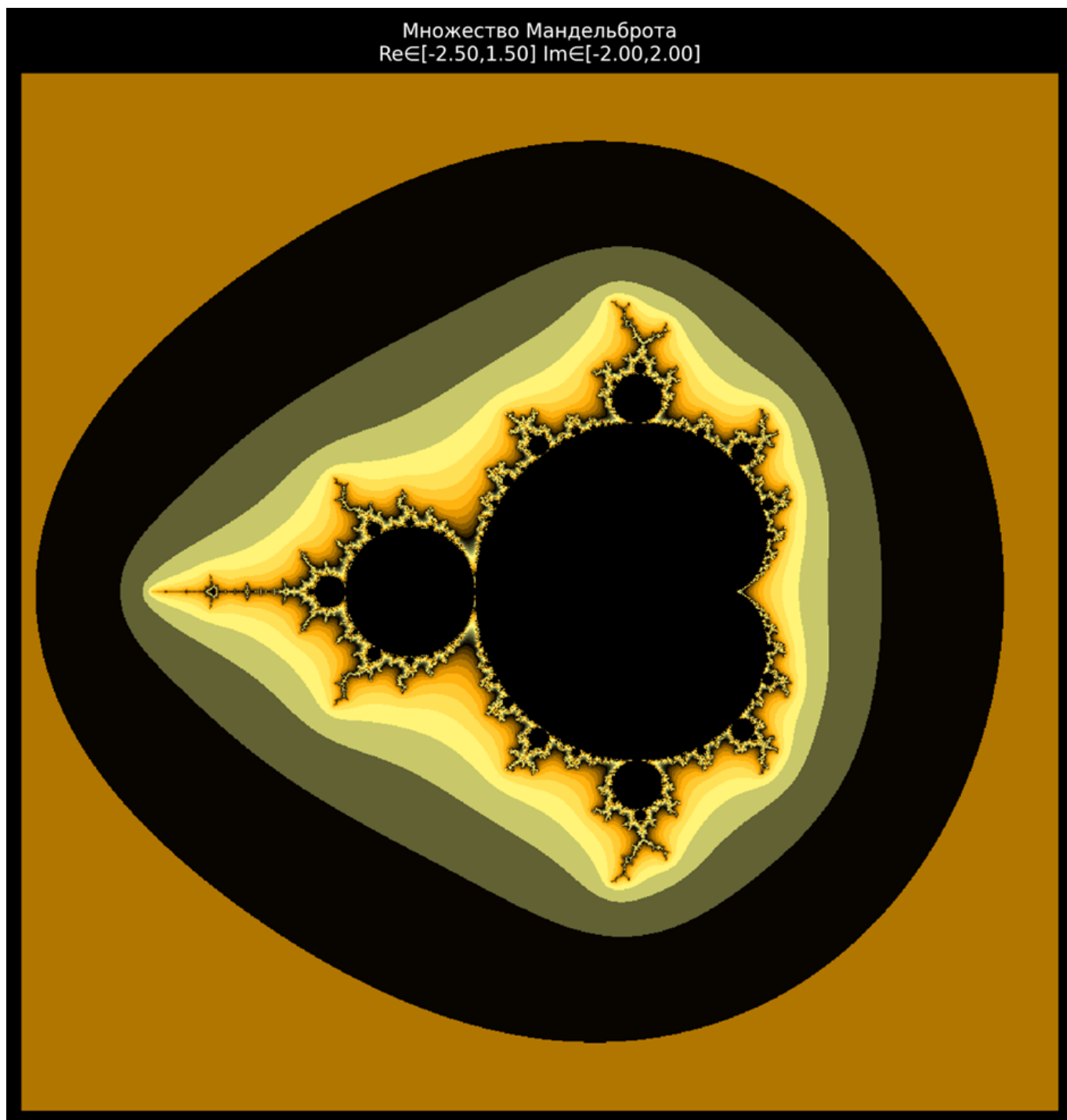
2.1 Код

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import matplotlib.colors as clr
4 from itertools import cycle
5 import os
6
7 def mandelbrot(pmin, pmax, qmin, qmax, ppoints=1000, qpoints=1000,
8               max_iterations=300, infinity_border=10):
9     image = np.zeros((ppoints, qpoints))
10    p, q = np.mgrid[pmin:pmax:(ppoints*1j), qmin:qmax:(qpoints*1j)]
11    c = p + 1j*q
12    z = np.zeros_like(c)
13    for k in range(max_iterations):
14        z = z**2 + c
15        mask = (np.abs(z) > infinity_border) & (image == 0)
16        image[mask] = k
17        z[mask] = np.nan
18    return -image.T
19
20 def make_colormap():
21     colorpoints = [
22         (1 - (1 - q)**4, c)
23         for q, c in zip(
24             np.linspace(0, 1, 20),
25             cycle(['#ffff88', '#000000', '#ffaa00'])
26         )
27     ]
28     return clr.LinearSegmentedColormap.from_list('mycmap', colorpoints, N=2048)
29
30 def render(image, bounds, cmap, filename):
31     """Отрисовка и сохранение фрактала"""
32     pmin, pmax, qmin, qmax = bounds
33     fig, ax = plt.subplots(figsize=(8, 8))
34     ax.imshow(image, extent=(pmin, pmax, qmin, qmax), cmap=cmap, interpolation='
35         none')
36     ax.set_title(f"Множество Мандельброта\n R e [{pmin:.2f},{pmax:.2f}] I m [{
37         qmin:.2f},{qmax:.2f}]",
38                 color='white', fontsize=10)
39     ax.set_xticks([]); ax.set_yticks([])
40     ax.set_facecolor('black')
41     plt.tight_layout()
42     os.makedirs("tfkp", exist_ok=True)
43     plt.savefig(os.path.join("tfkp", filename), dpi=300, bbox_inches='tight',
44                 facecolor='black')
45     plt.close(fig)
46     print(f"Сохранено: tfkp/{filename}")
47
48 if __name__ == "__main__":
49     cmap = make_colormap()
50     full_bounds = (-2.5, 1.5, -2.0, 2.0)
51     image = mandelbrot(*full_bounds, 1000, 1000, max_iterations=300)
52     render(image, full_bounds, cmap, "mandelbrot_full.png")
53     zooms = [
54         (-0.75, -0.70, 0.05, 0.10),
55         (-1.25, -1.15, 0.20, 0.30),
56         (0.25, 0.35, -0.65, -0.55),
```

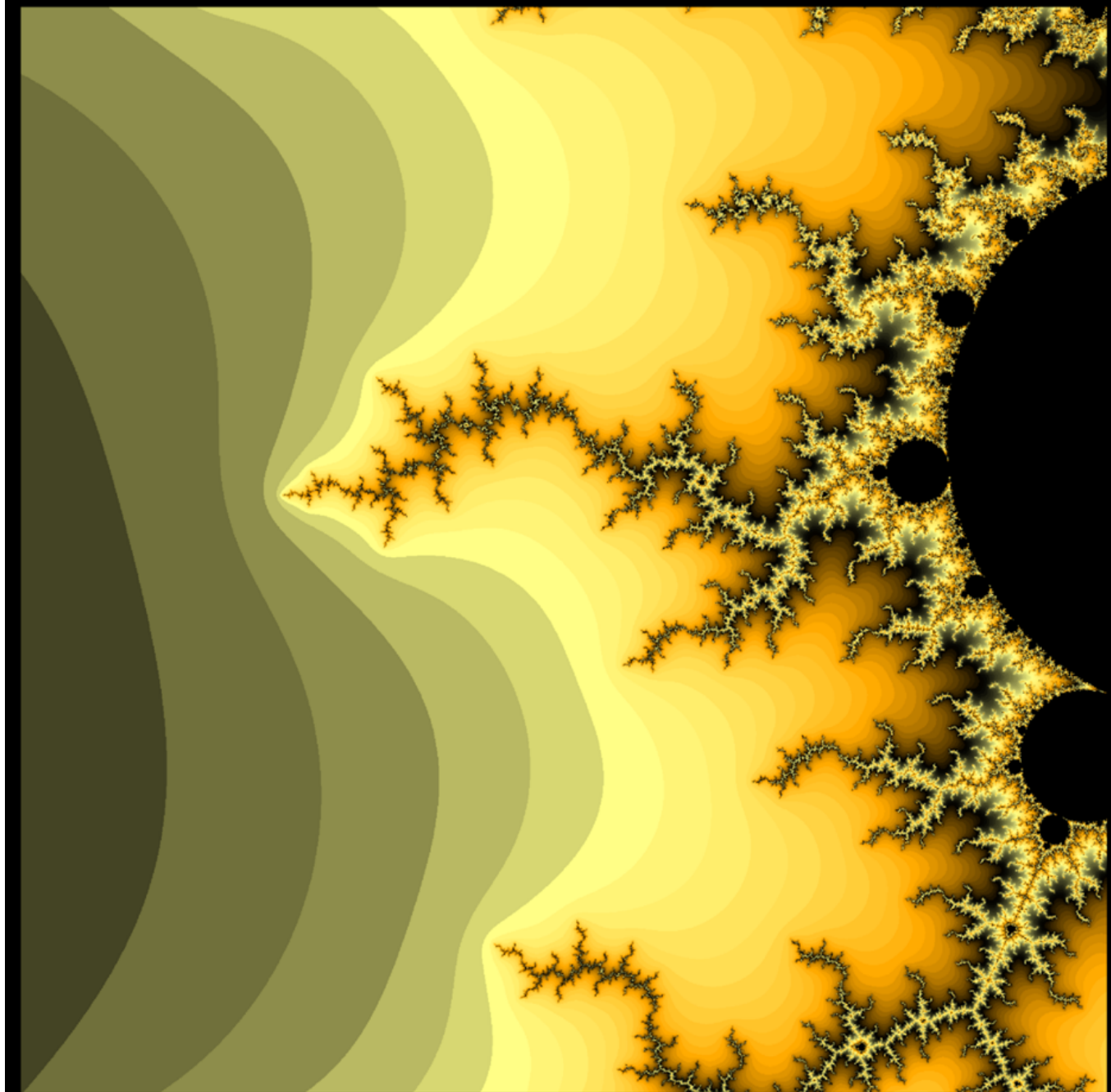
```
54     (-1.40, -1.20, -0.05, 0.05),  
55     (0.35, 0.45, 0.35, 0.45)  
56 ]  
57  
58 for i, b in enumerate(zooms, start=1):  
59     img_zoom = mandelbrot(*b, 1500, 1500, max_iterations=800)  
60     render(img_zoom, b, cmap, f"mandelbrot_zoom_{i}.png")  
61  
62 print("Все изображения сохранены")
```

2.2 Картинки

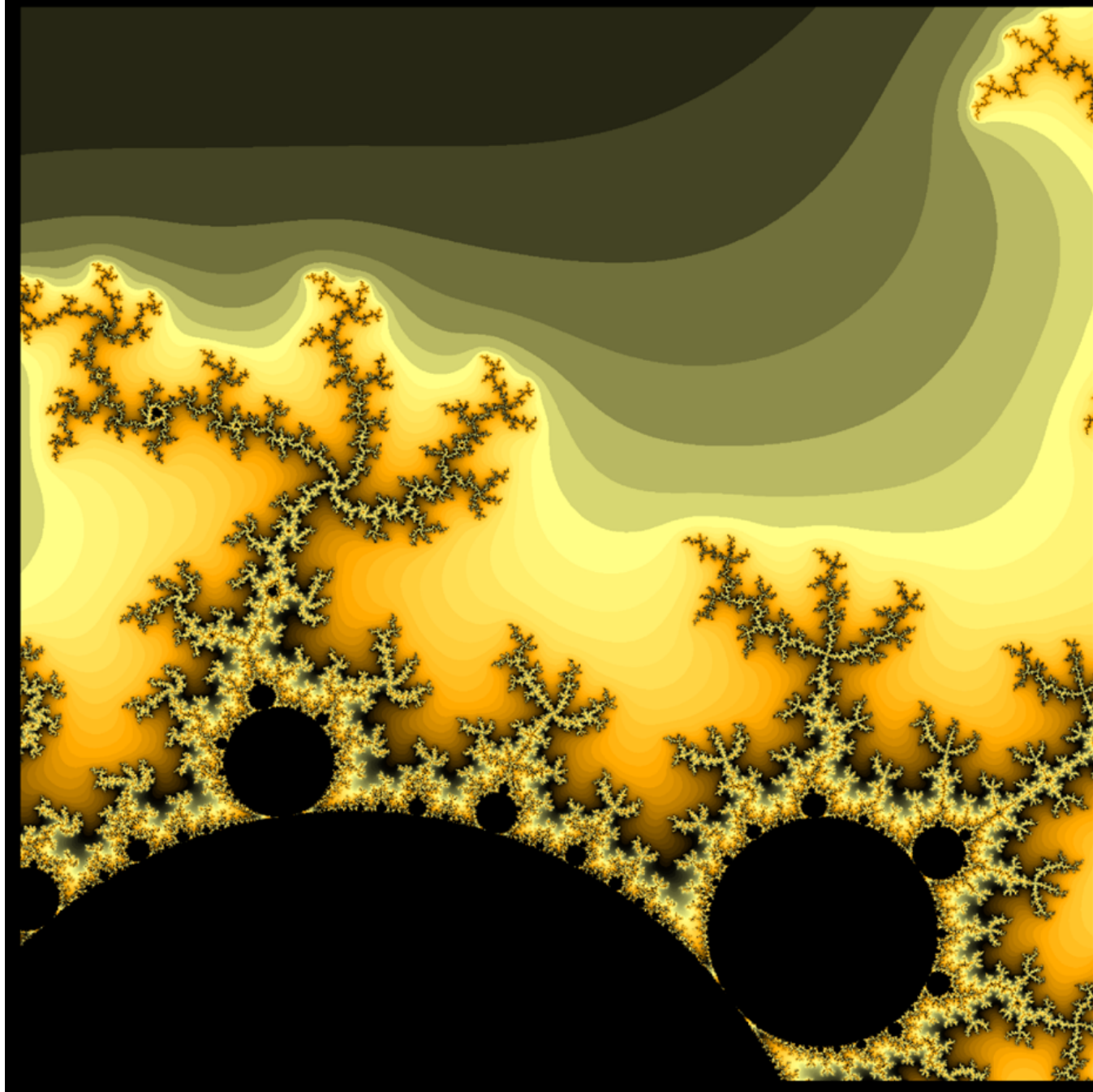
Множество Мандельброта
 $\text{Re} \in [-2.50, 1.50]$ $\text{Im} \in [-2.00, 2.00]$



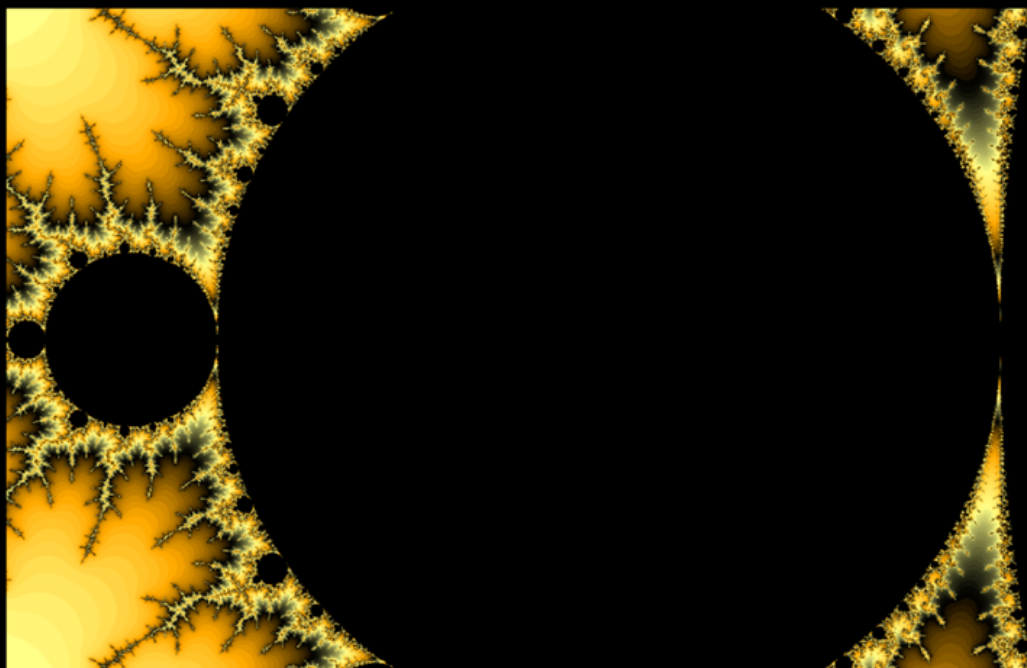
Множество Мандельброта
 $\text{Re} \in [-1.25, -1.15]$ $\text{Im} \in [0.20, 0.30]$



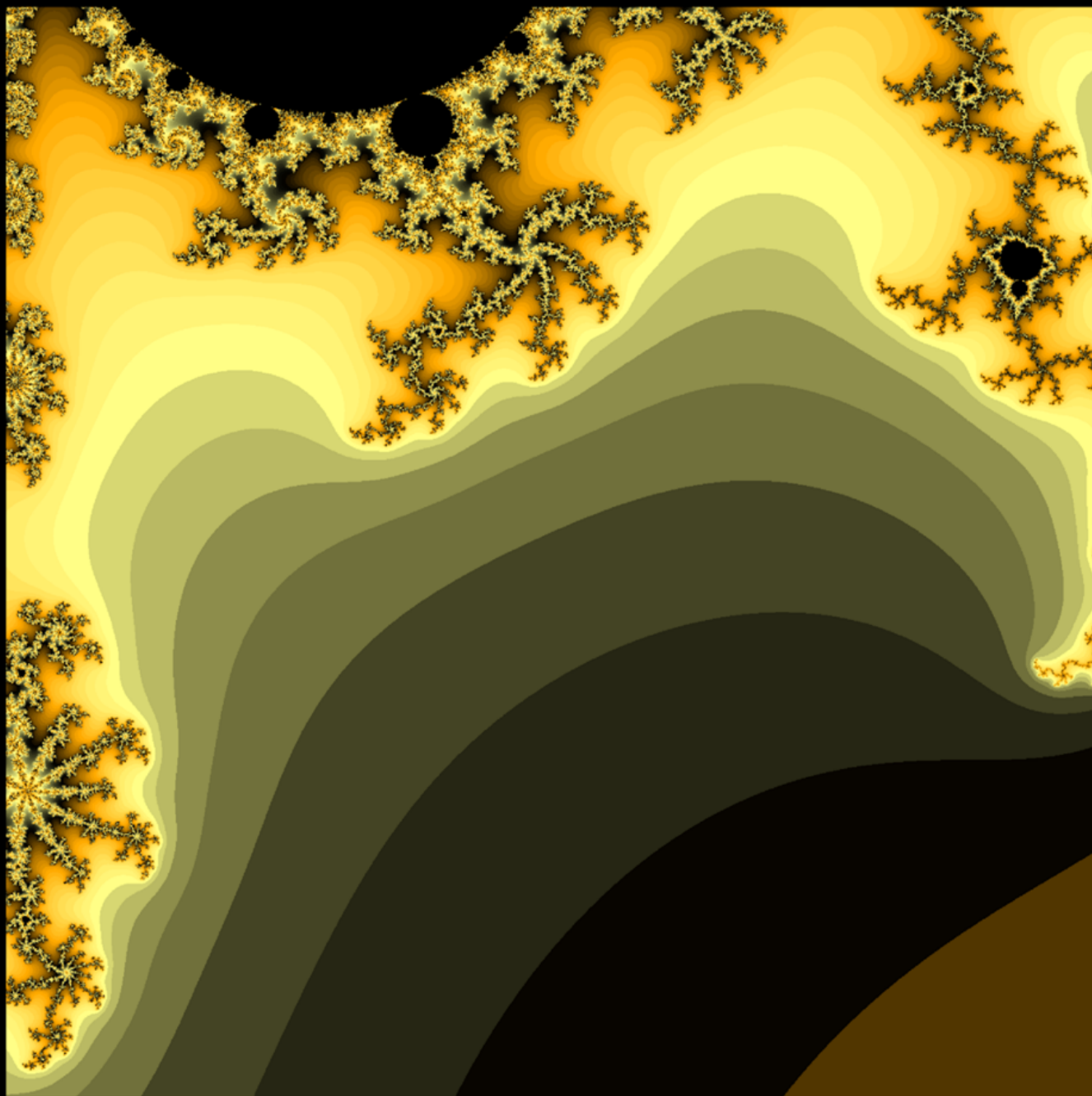
Множество Мандельброта
 $\text{Re} \in [0.25, 0.35] \text{ Im} \in [-0.65, -0.55]$



Множество Мандельброта
 $\text{Re} \in [-1.40, -1.20]$ $\text{Im} \in [-0.05, 0.05]$



Множество Мандельброта
 $\text{Re} \in [0.35, 0.45]$ $\text{Im} \in [0.35, 0.45]$



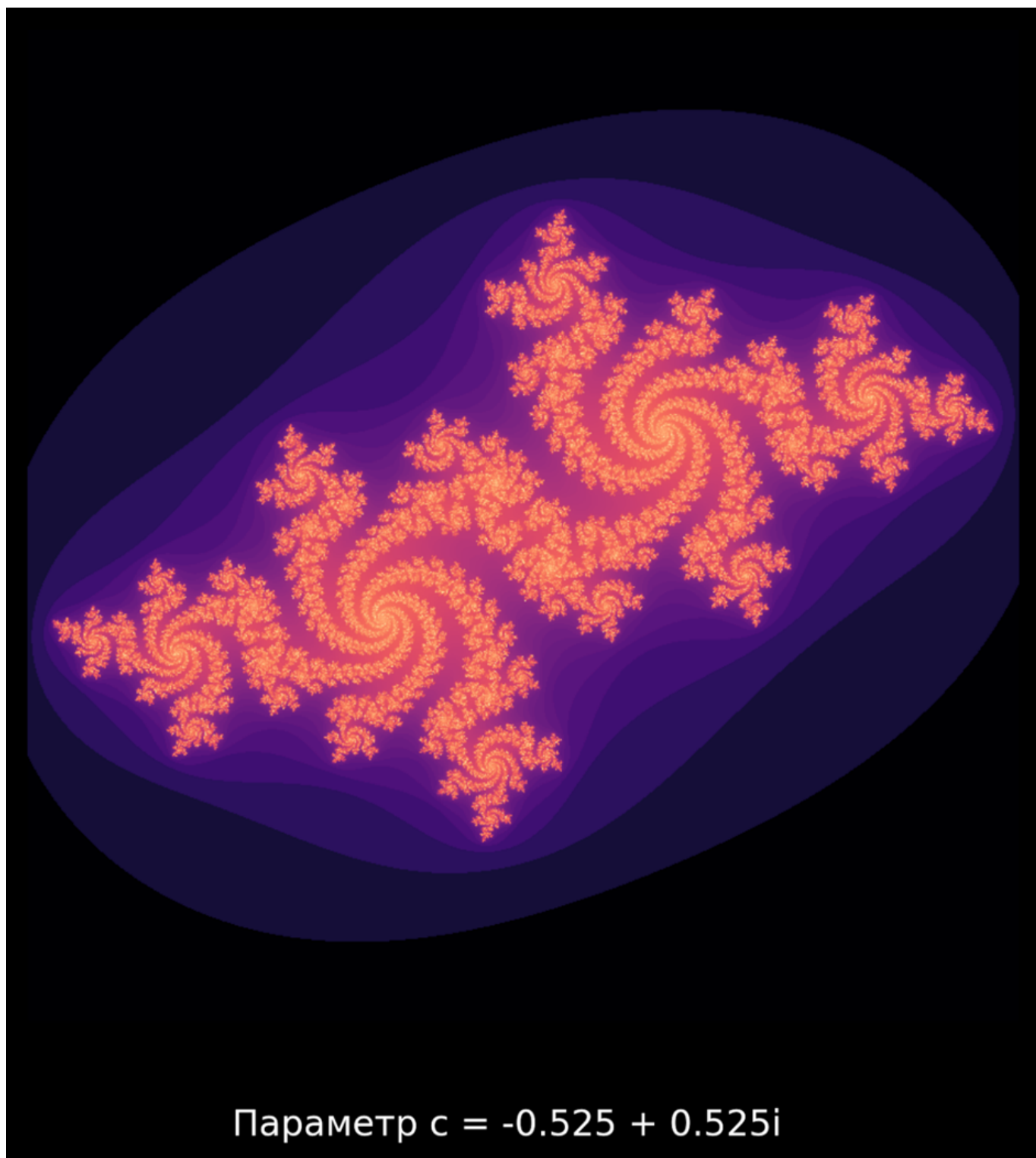
3 Визуализация заполненного множества Жюлиа

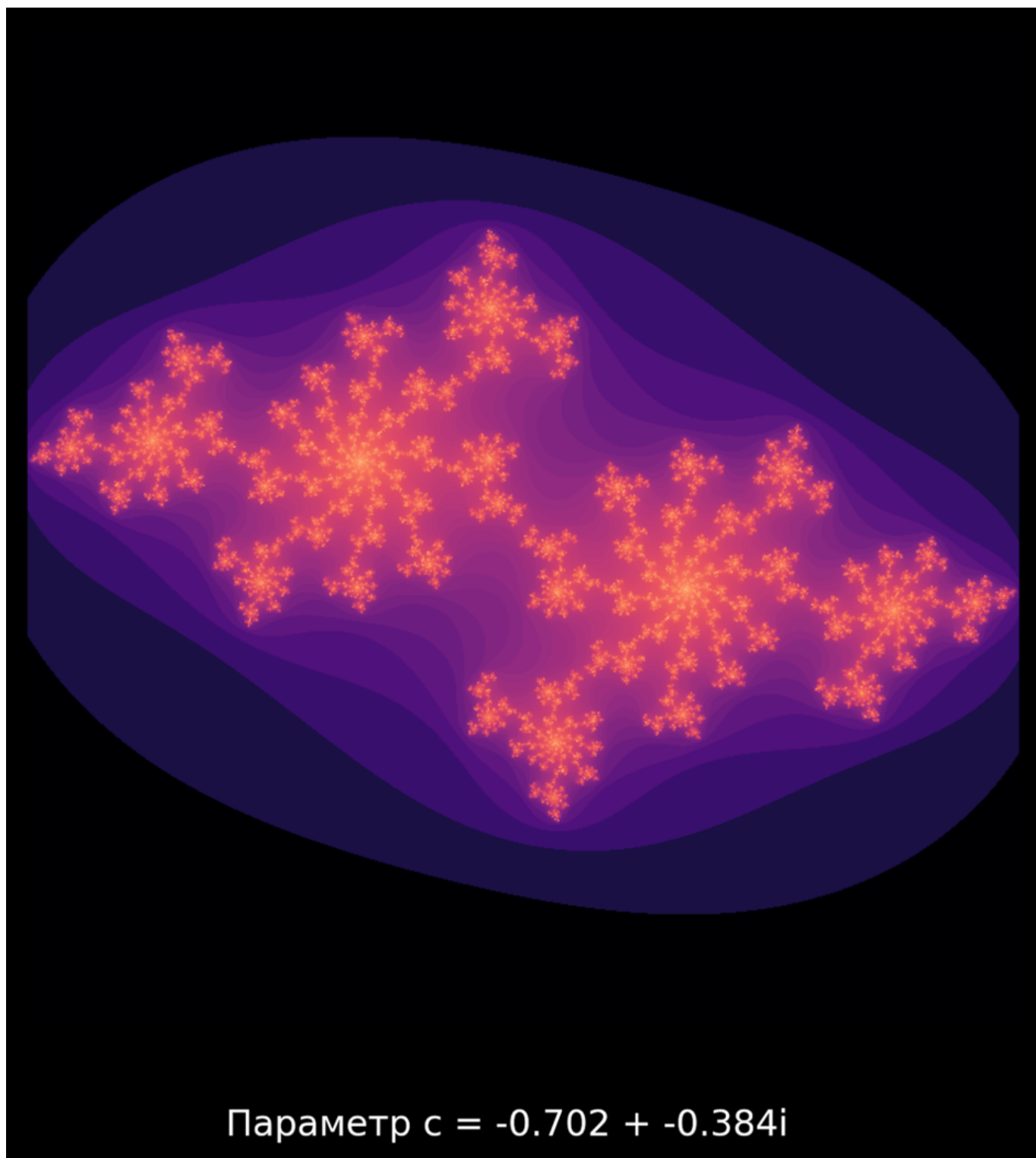
3.1 Код

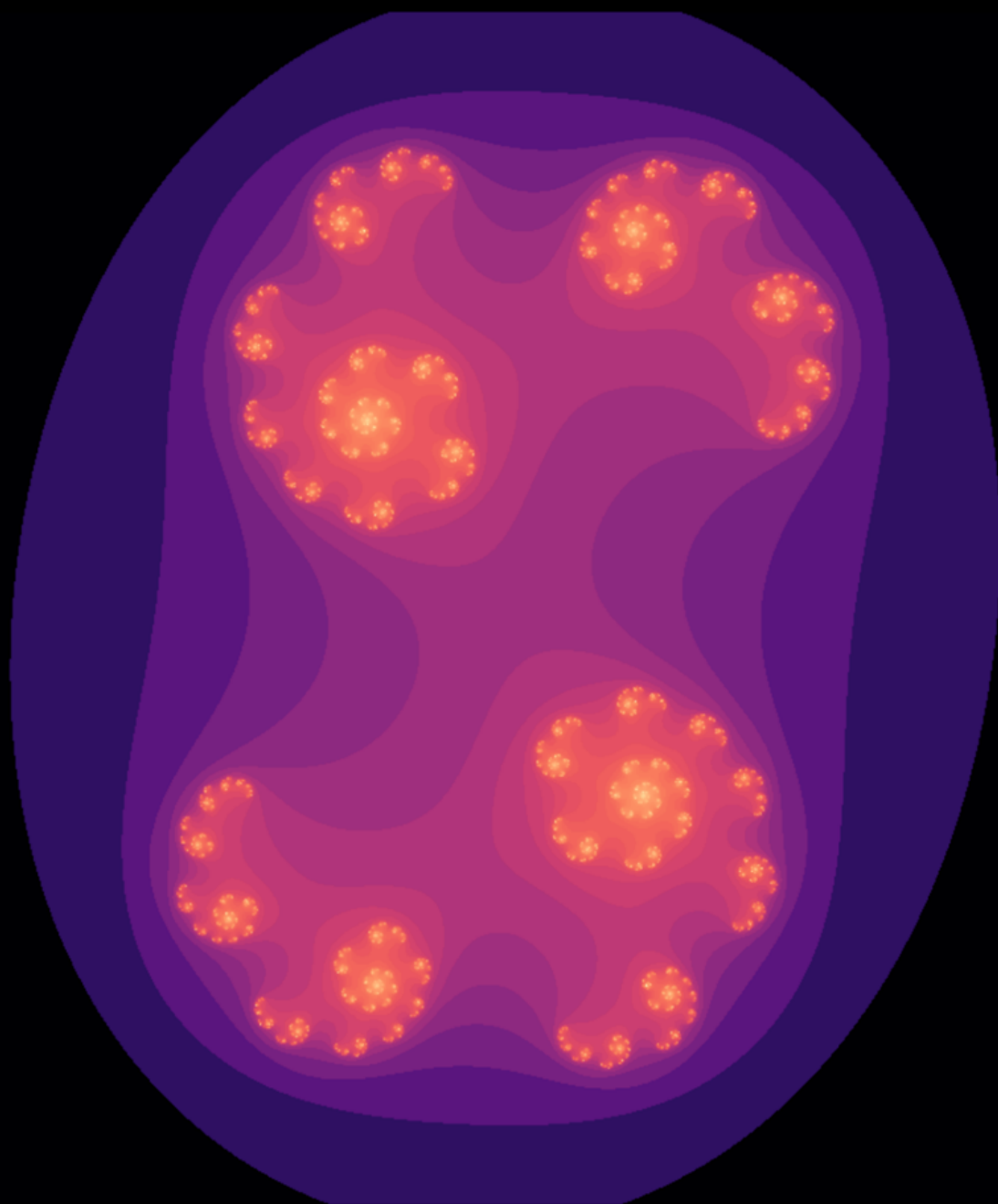
```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import os
4 from time import time
5
6 WIDTH = HEIGHT = 800
7 MAX_ITER = 10000
8 ESCAPE_RADIUS = 4.0
9
10 def compute_julia(c, bounds=(-1.5, 1.5), resolution=(WIDTH, HEIGHT)):
11     x_min, x_max = bounds
12     y_min, y_max = bounds
13     cols, rows = resolution
14
15     x = np.linspace(x_min, x_max, cols)
16     y = np.linspace(y_min, y_max, rows)
17     X, Y = np.meshgrid(x, y)
18     Z = X + 1j * Y
19     counts = np.zeros(Z.shape, dtype=np.int32)
20     active = np.ones(Z.shape, dtype=bool)
21
22     for i in range(MAX_ITER):
23         Z[active] = Z[active] ** 2 + c
24         mask = (Z.real**2 + Z.imag**2) > ESCAPE_RADIUS
25         escaped = mask & active
26         counts[escaped] = i
27         active &= ~escaped
28         if not np.any(active):
29             break
30
31     counts[active] = MAX_ITER
32     return counts
33
34 def render_julia(image, c, output_path):
35     normalized = np.log1p(image)
36     normalized /= normalized.max()
37
38     fig, ax = plt.subplots(figsize=(6, 6), facecolor='black')
39     ax.imshow(normalized, extent=(-1.5, 1.5, -1.5, 1.5), cmap='magma')
40     ax.axis('off')
41     fig.text(0.5, 0.03, f"Параметр c = {c.real:.3f} + {c.imag:.3f}i",
42             ha='center', va='center', color='white', fontsize=12)
43     plt.savefig(output_path, dpi=300, bbox_inches='tight')
44     plt.close(fig)
45
46 if __name__ == "__main__":
47     output_dir = "tfkp"
48     os.makedirs(output_dir, exist_ok=True)
49
50     constants = [
51         -0.5251993 + 0.5251993j,
52         -0.70176 - 0.3842j,
53         0.45 + 0.1428j,
54         -0.8 + 0.156j,
55         0.0 + 0.0j
56     ]
```

```
57
58  for idx, c in enumerate(constants, start=1):
59      t0 = time()
60      fractal = compute_julia(c)
61      filename = f"julia_{idx}_{c.real:.3f}_{c.imag:.3f}.png"
62      render_julia(fractal, c, os.path.join(output_dir, filename))
63      print(f"Сделано {filename} за {time() - t0:.1f} сек.")
```

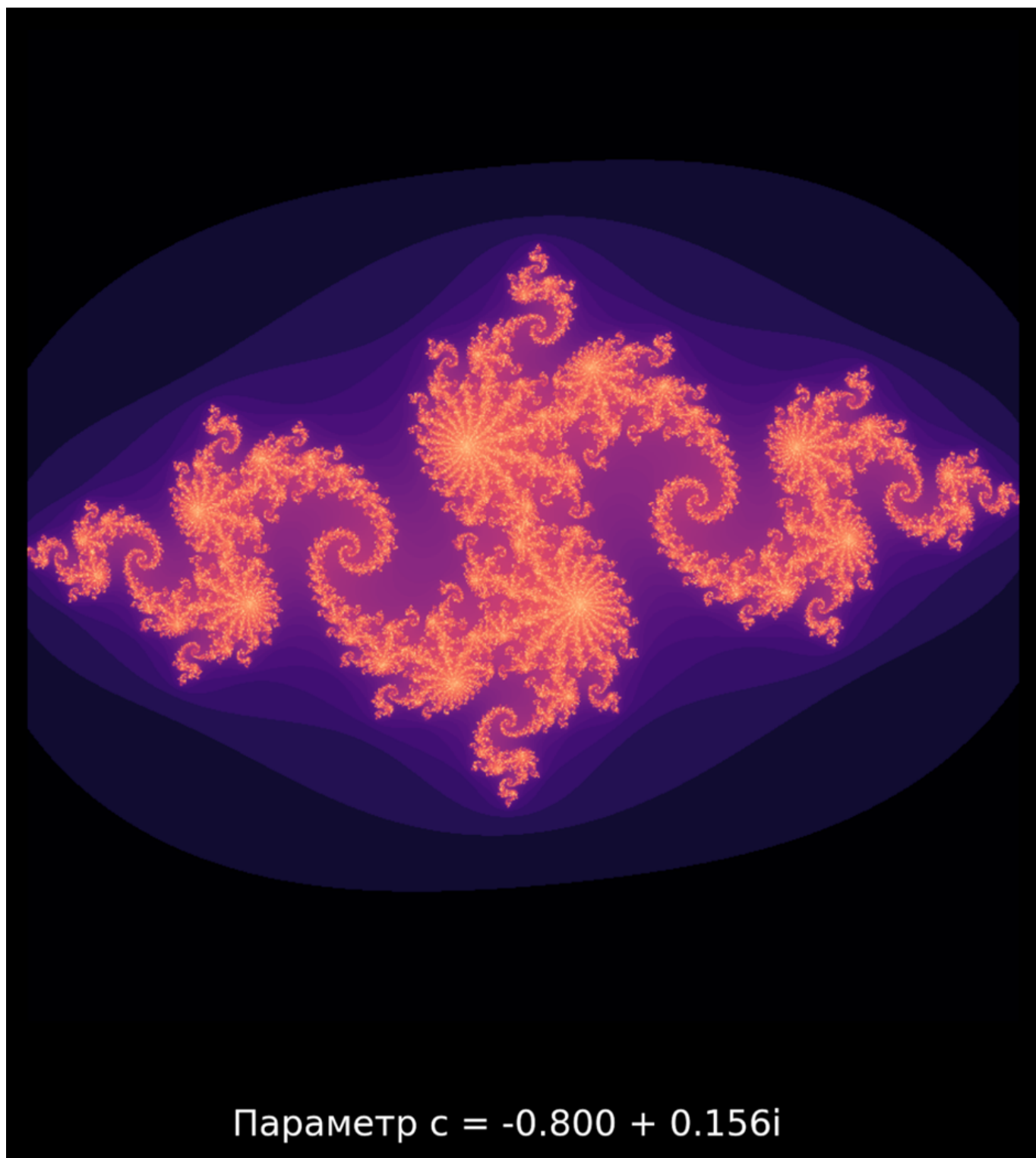
3.2 Картинки

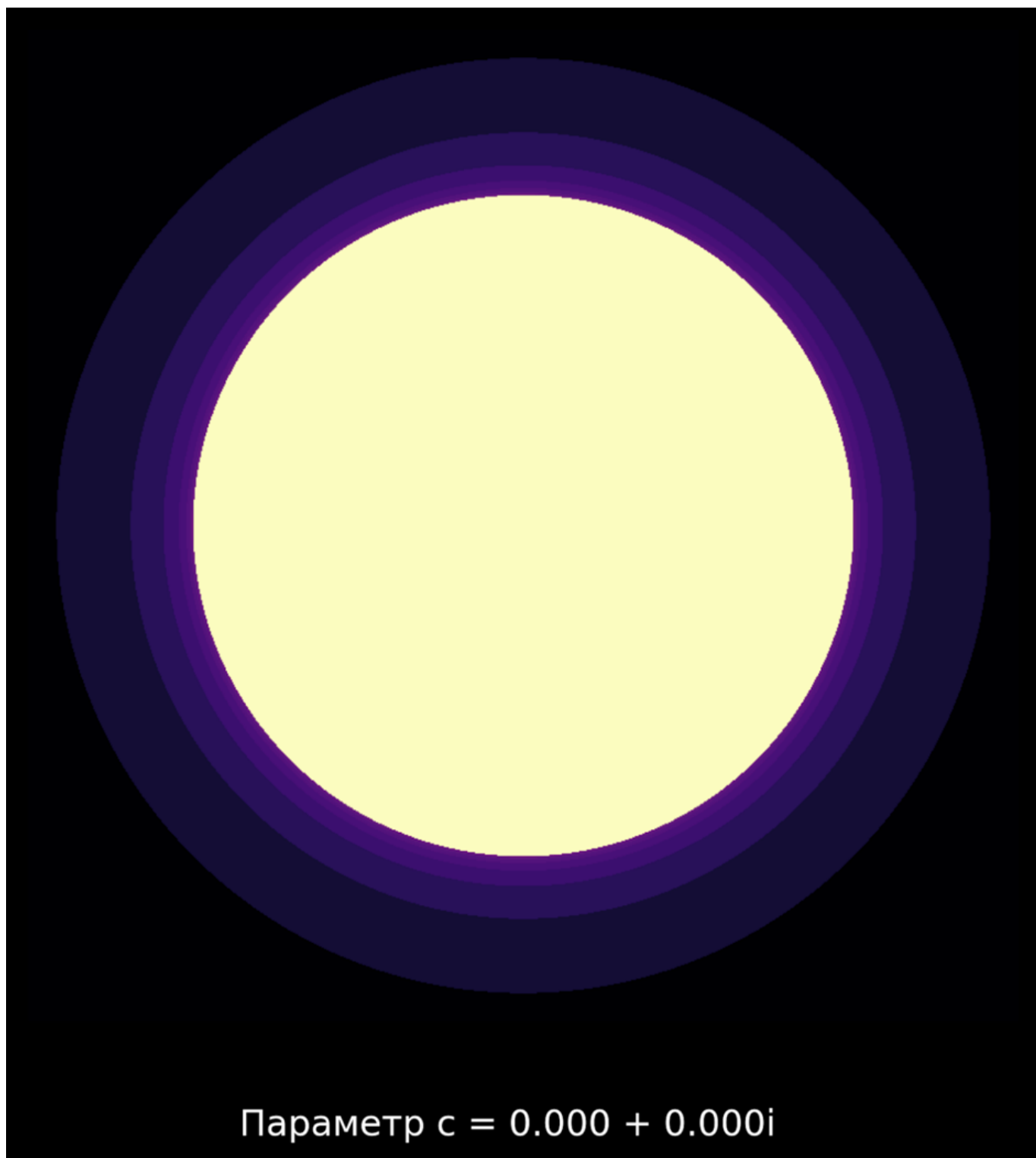






Параметр $c = 0.450 + 0.143i$





4 Magnet fractals Type II

4.1 Описание

Магнитный фрактал — это концепция, описывающая структуру магнитных полей как фрактальную, то есть обладающую свойством самоподобия на разных масштабах. Исторически фрактальные магнитные поля исследовались в научных работах, например, у Эпштейна и Миллера, которые создали фрактальные магнитные поля в искусственных материалах, обнаружив, что форма магнитного поля в таких материалах не укладывается в традиционные трехмерные модели, а характеризуется дробной размерностью (около 0,8-1,6 мерности)

$$z_{n+1} = \left[\frac{z_n^3 + 3(c-1)z_n + (c-1)(c-2)}{3z_n^2 + 3(c-2)z_n + (c-1)(c-2)} \right]^2$$

4.2 Код

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import matplotlib.colors as clr
4 from itertools import cycle
5 import os
6
7 def magnet_fractal(pmin, pmax, qmin, qmax, ppoints=1000, qpoints=1000,
8                   max_iterations=300, infinity_border=40):
9     image = np.zeros((ppoints, qpoints))
10    p, q = np.mgrid[pmin:pmax:(ppoints*1j), qmin:qmax:(qpoints*1j)]
11    c = p + 1j*q
12    z = np.zeros_like(c)
13    mask = np.ones_like(c, dtype=bool)
14    for k in range(1, max_iterations+1):
15        with np.errstate(divide='ignore', invalid='ignore'):
16            numerator = z**3 + 3*(c-1)*z + (c-1)*(c-2)
17            denominator = 3*z**2 + 3*(c-2)*z + (c-1)*(c-2) + 1
18            valid = denominator != 0
19            # Изменяем только те точки, где делитель не ноль
20            z_next = np.where(valid, (numerator/denominator)**2, z)
21            z = np.where(valid, z_next, z)
22        escape = (np.abs(z) > infinity_border) & mask
23        image[escape] = k
24        mask[escape] = False
25        if not np.any(mask): # Все вышли за границу
26            break
27    image[image == 0] = max_iterations
28    return -image.T
29
30 def make_colormap():
31     colorpoints = [
32         (1 - (1 - q)**4, c)
33         for q, c in zip(
34             np.linspace(0, 1, 20),
35             cycle(['#ffff88', '#000000', '#ffaa00'])
36         )
37     ]
38     return clr.LinearSegmentedColormap.from_list('mycmap', colorpoints, N=2048)
39
40 def render(image, bounds, cmap, filename):
41     pmin, pmax, qmin, qmax = bounds
```

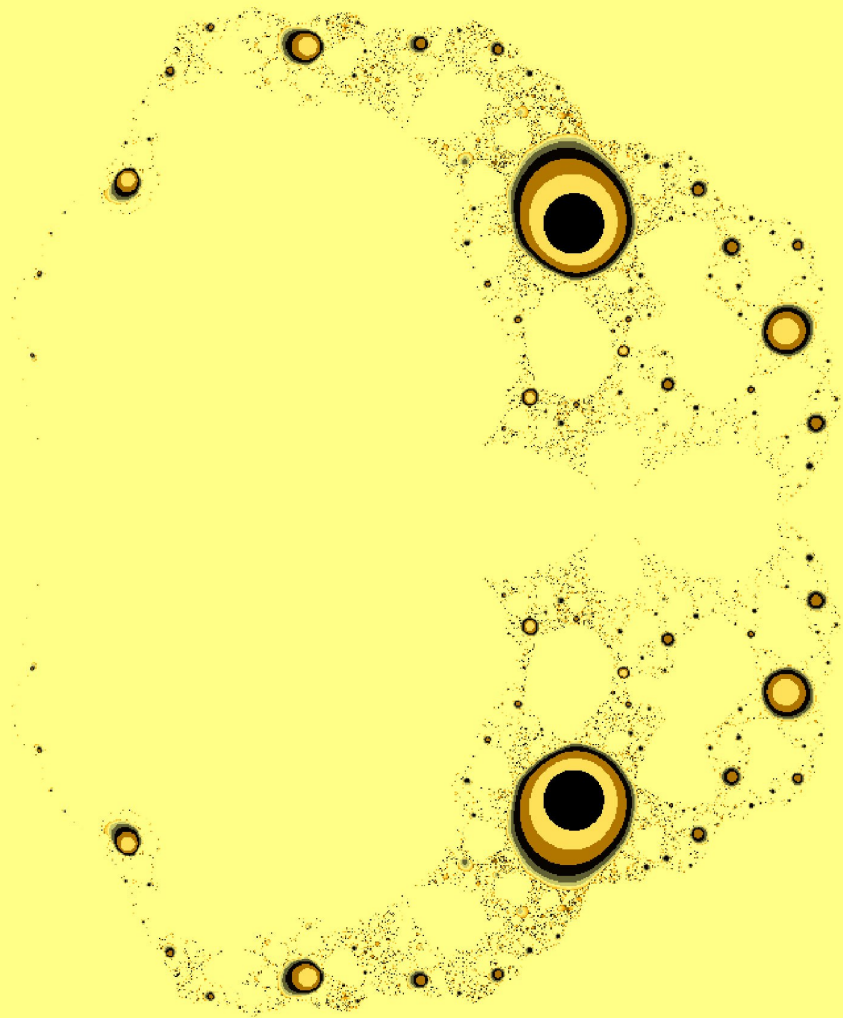
```

42 fig, ax = plt.subplots(figsize=(8, 8))
43 ax.imshow(image, extent=(pmin, pmax, qmin, qmax), cmap=cmap, interpolation='
    none')
44 ax.set_title(f"Magnet Fractal 2\ n R e [{pmin:.2f},{pmax:.2f}] I m [{qmin:.2f
    },{qmax:.2f}]",
45             color='white', fontsize=10)
46 ax.set_xticks([]); ax.set_yticks([])
47 ax.set_facecolor('black')
48 plt.tight_layout()
49 os.makedirs("tfkp", exist_ok=True)
50 plt.savefig(os.path.join("tfkp", filename), dpi=300, bbox_inches='tight',
    facecolor='black')
51 plt.close(fig)
52 print(f"Сохранено: tfkp/{filename}")
53
54 if __name__ == "__main__":
55     cmap = make_colormap()
56     full_bounds = (0.75, 1, 1.25, 1.5)
57     image = magnet_fractal(*full_bounds, 1000, 1000, max_iterations=300)
58     render(image, full_bounds, cmap, "magnet4_full.png")

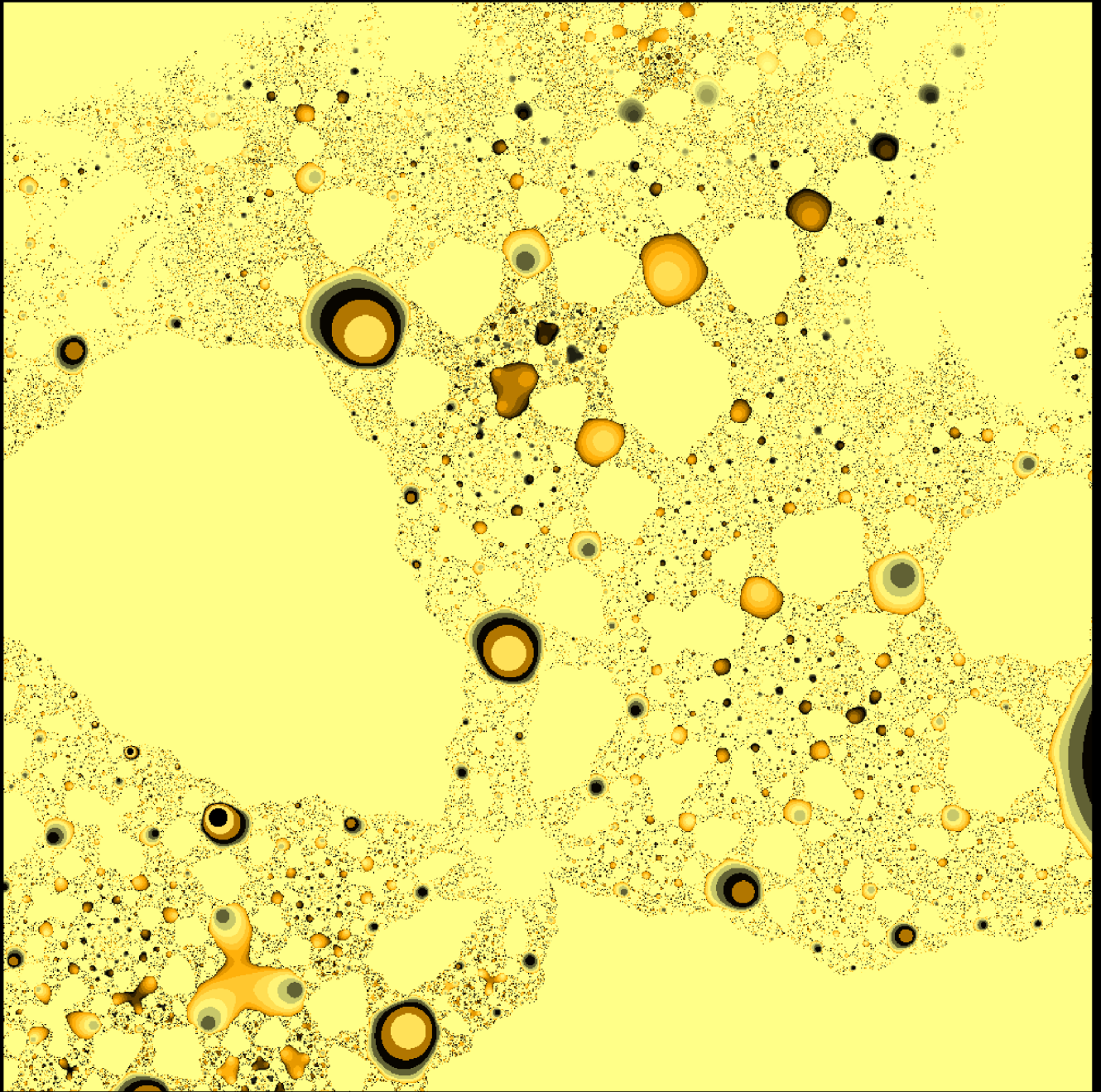
```

4.3 Картинки

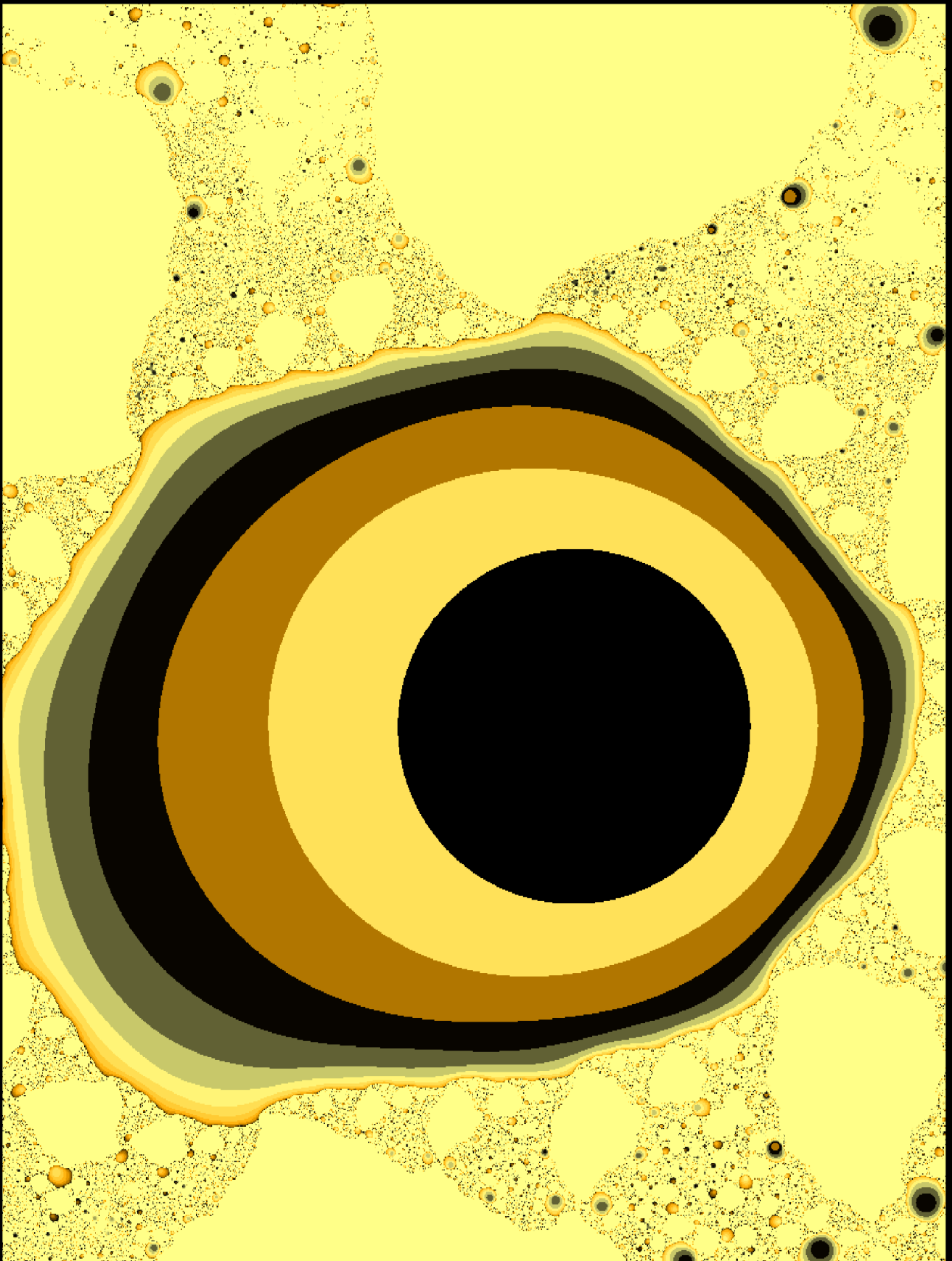
Magnet Fractal 2
 $\text{Re} \in [-1.50, 2.50]$ $\text{Im} \in [-2.00, 2.00]$



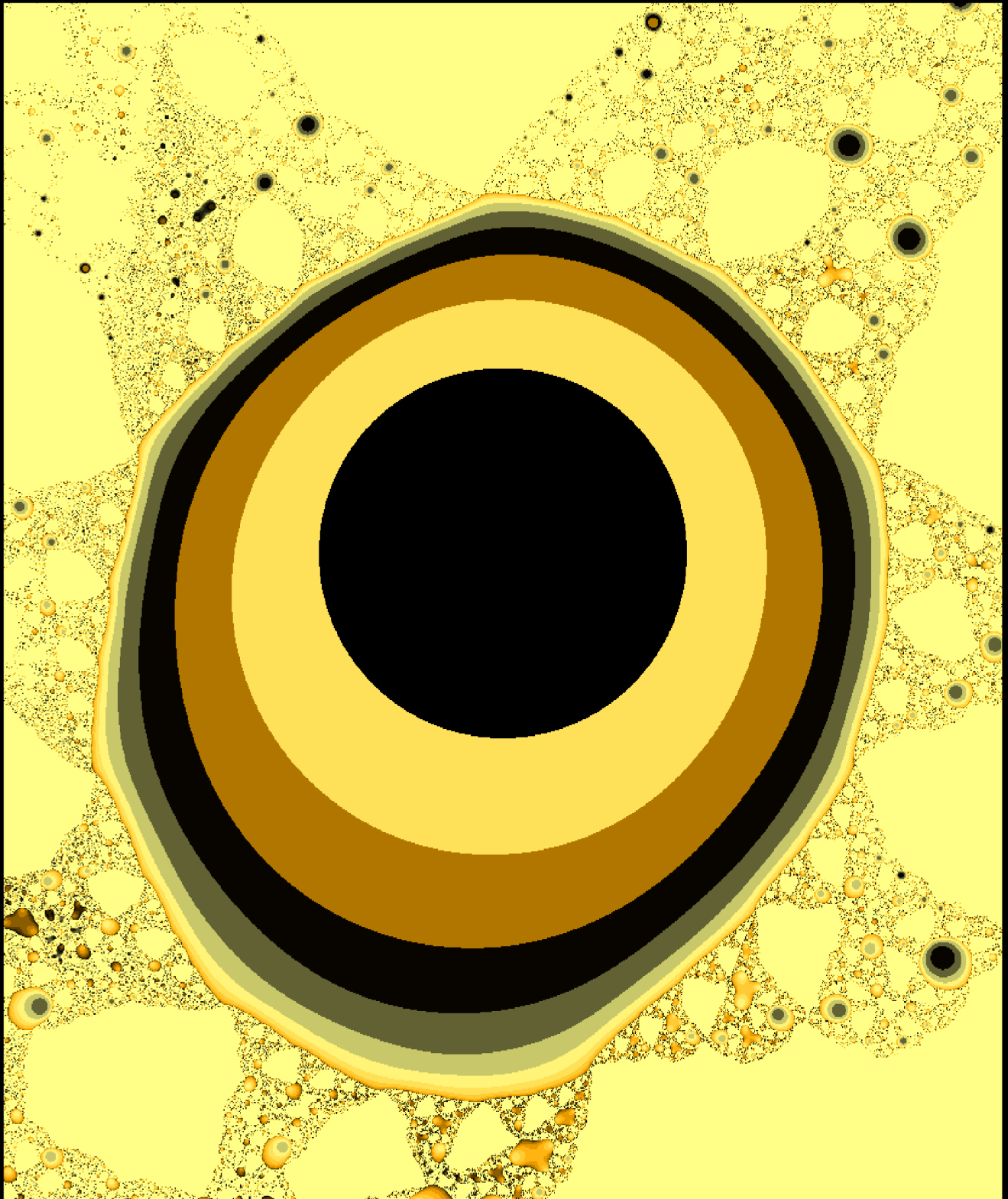
Magnet Fractal 2
 $\text{Re} \in [0.75, 1.00]$ $\text{Im} \in [1.25, 1.50]$



Magnet Fractal 2
 $\text{Re} \in [0.60, 0.75]$ $\text{Im} \in [1.30, 1.50]$



Magnet Fractal 2
 $\text{Re} \in [1.25, 1.75]$ $\text{Im} \in [0.60, 1.20]$



5 Вывод

В команде мы на практике узнали как программно задавать и строить графики фракталов и какое у них есть физическое отображение.